SEMI SUPERVISED LEARNING AND REINFORCEMENT LEARNING

Mrs. Jisha Tinsu
Assistant Professor
Dept. of Information Technology, TCET

Contents.....

- Semi-supervised Learning
- Benefits and limitations
- Semi-supervised Learning Techniques
- Reinforcement Learning: Basics, Elements, Algorithms
- Use cases

Semi Supervised learning

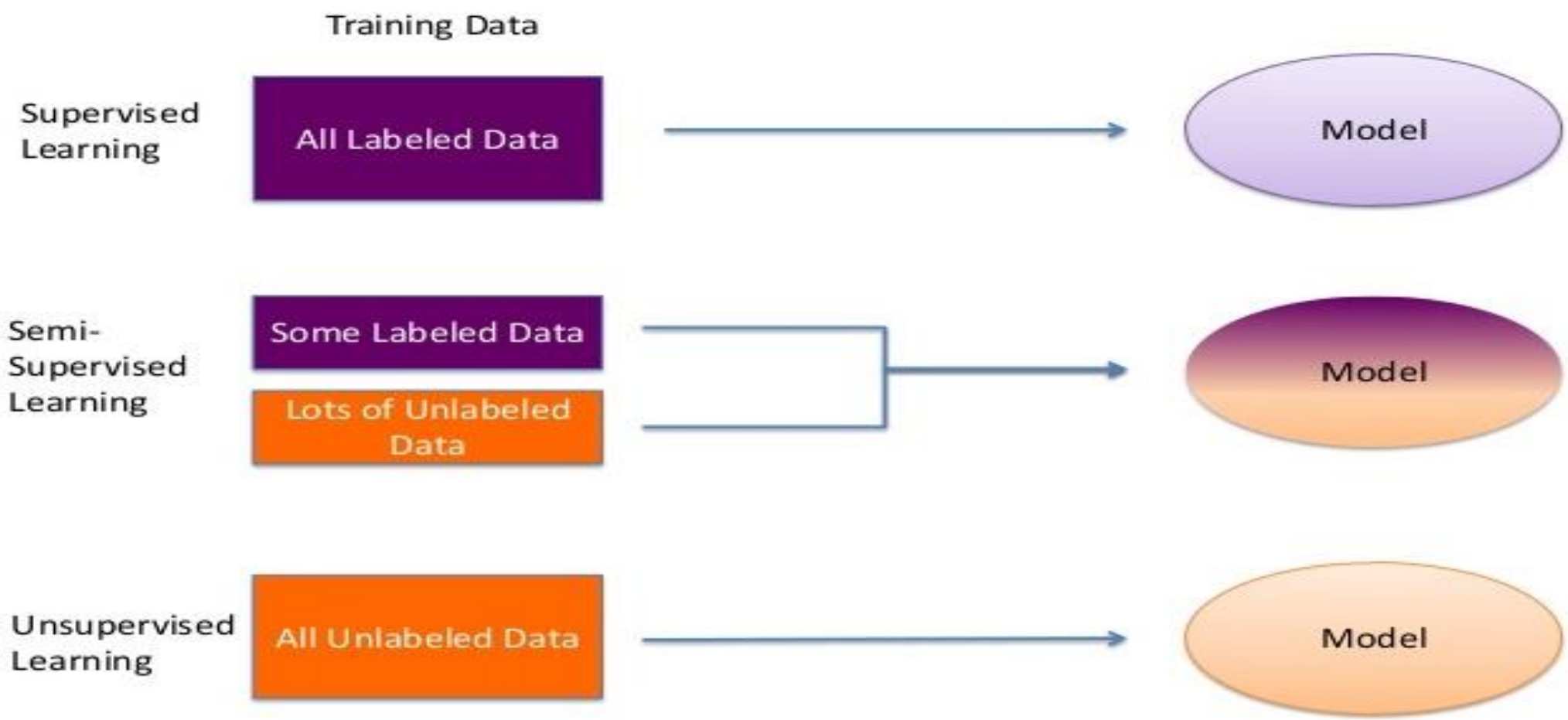
A human brain does not require millions of data for training with multiple iterations of going through the same image for understanding a topic. All it needs is a few guiding points to train itself on the underlying patterns. Clearly, we are missing something in current machine learning approaches.

Semi Supervised learning

- Steps:
- 1) Train model with these two images and test
- 2) Efficiency is low as dataset is poor
- 3) Download few more images from the Google
- 4) Increase the dataset and label all images
- 5) Check model performance

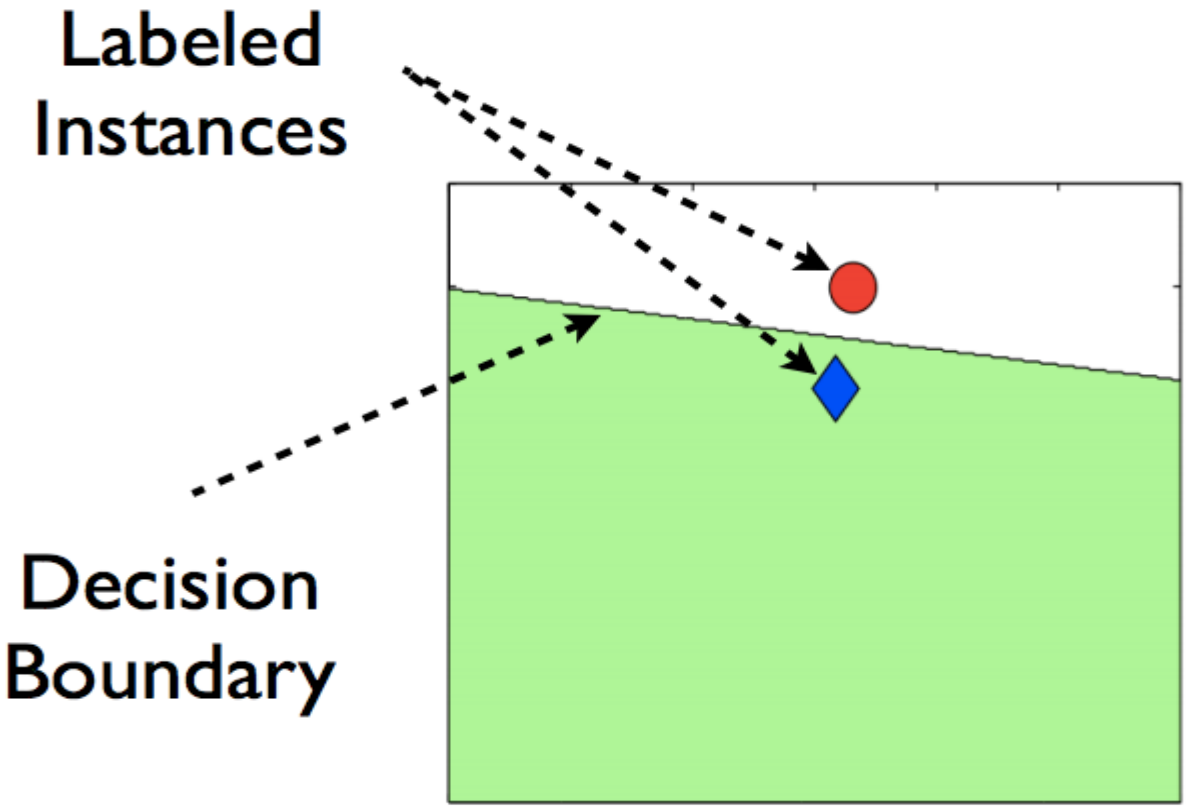
Difficulty is manually labelling of images

Semi Supervised learning



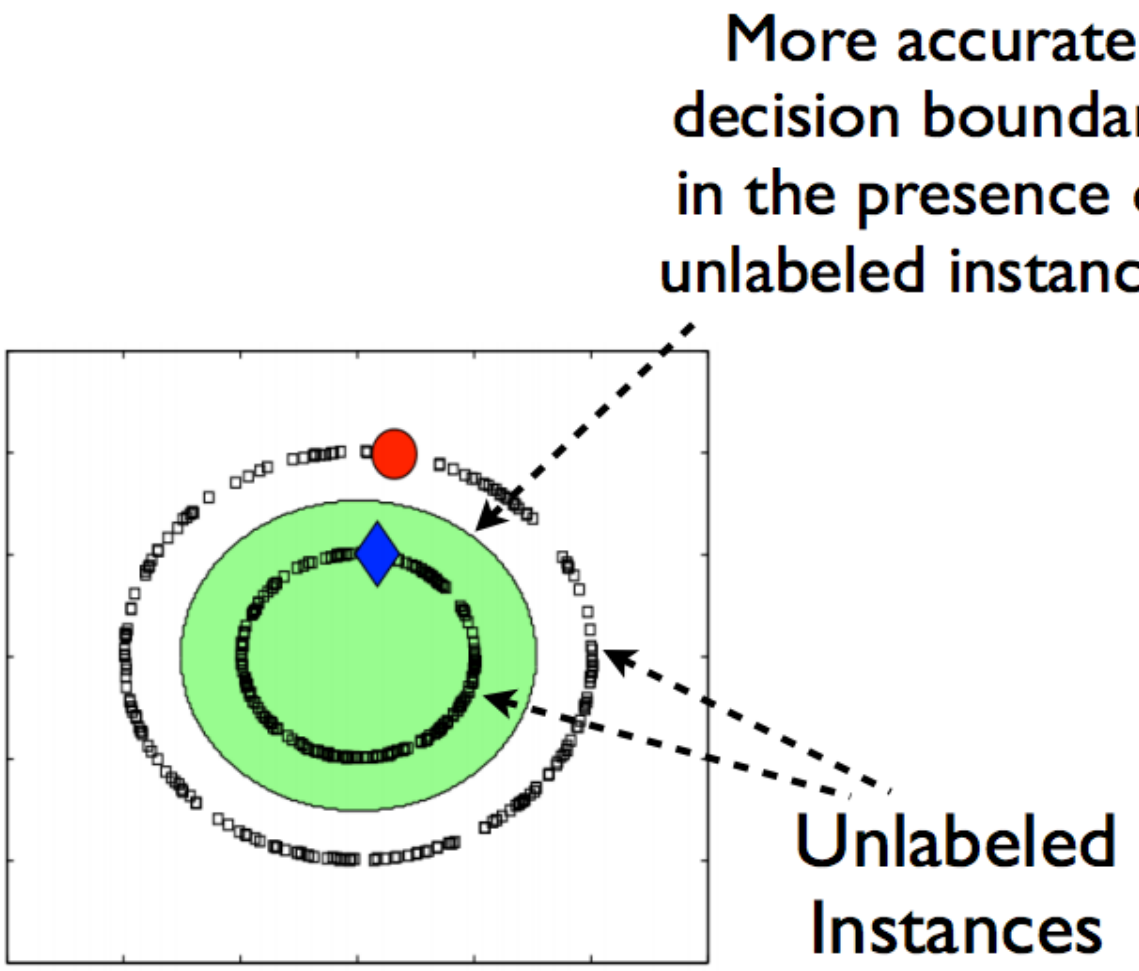
Semi Supervised learning

- Only two data points belonging to two different categories, and the line drawn is the decision boundary of any supervised model.



Semi Supervised learning

- add some unlabeled data to this data as shown in the image below.



If we notice the difference between the above two images, after adding unlabeled data, the decision boundary of model has become more accurate.

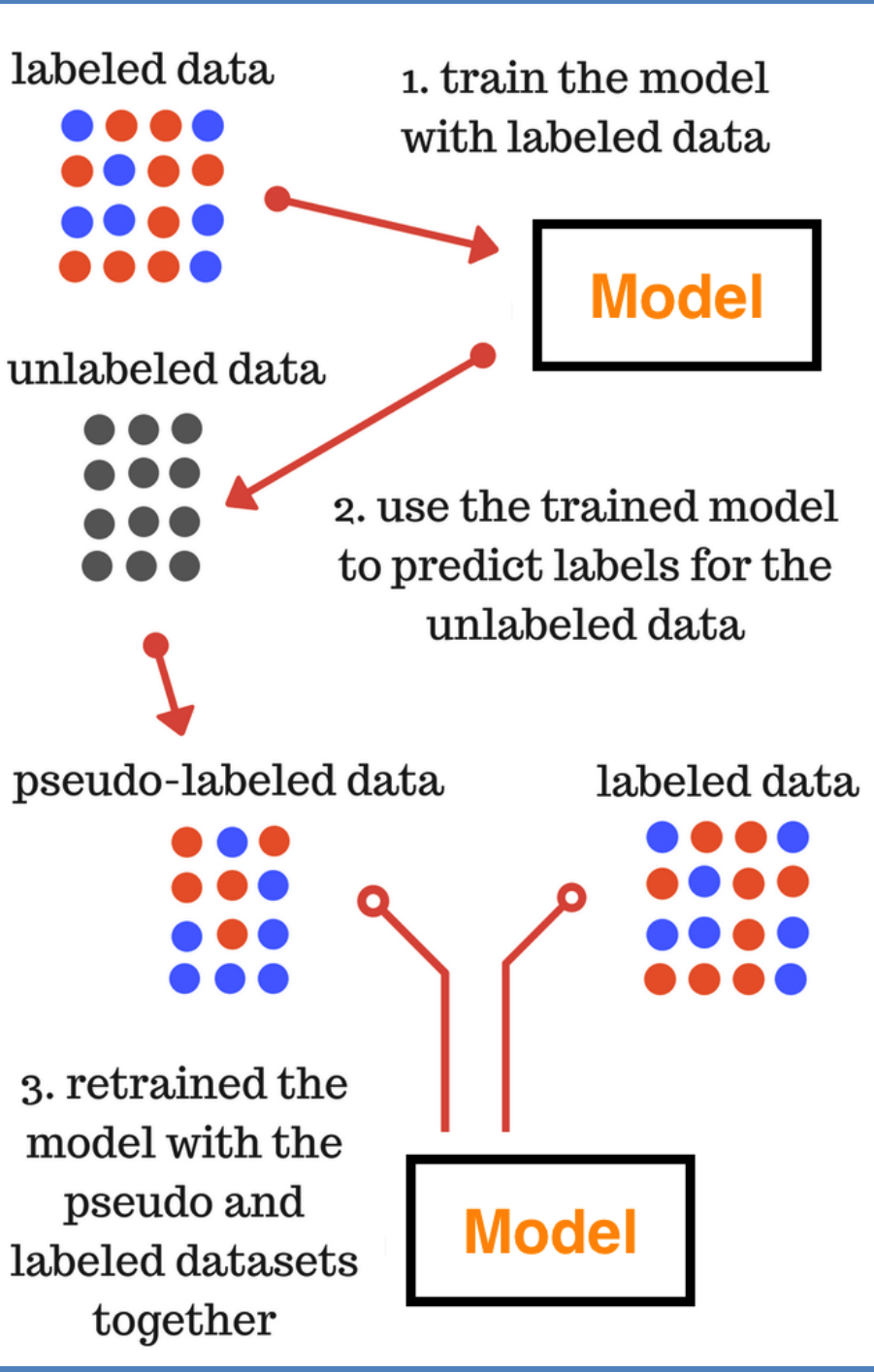
Semi Supervised learning

- So, the advantage of using unlabeled data are:

- Labelled data is expensive and difficult to get while unlabeled is abundant and cheap.
- It improves the model robustness by more precise decision boundary.

SSL-Pseudo Labelling

- **Pseudo-labeling** is a **self-training** SSL method where a model trained on labeled data is used to assign labels (called **pseudo-labels**) to the **unlabeled data**. Then this pseudo-labeled data is used to retrain or fine-tune the model.
- Final model trained in the third step is used for the final predictions on the test data.



How Pseudo-Labeling Works

Step-by-Step Process:

- **Train initial model** on small **labeled dataset**.
- **Predict labels** for the **unlabeled data**.
- **Select confident predictions** (usually using a confidence threshold).
- Treat those predictions as **pseudo-labels**.
- **Combine** labeled and pseudo-labeled data to retrain the model.
- Repeat for multiple iterations to improve performance.

Example

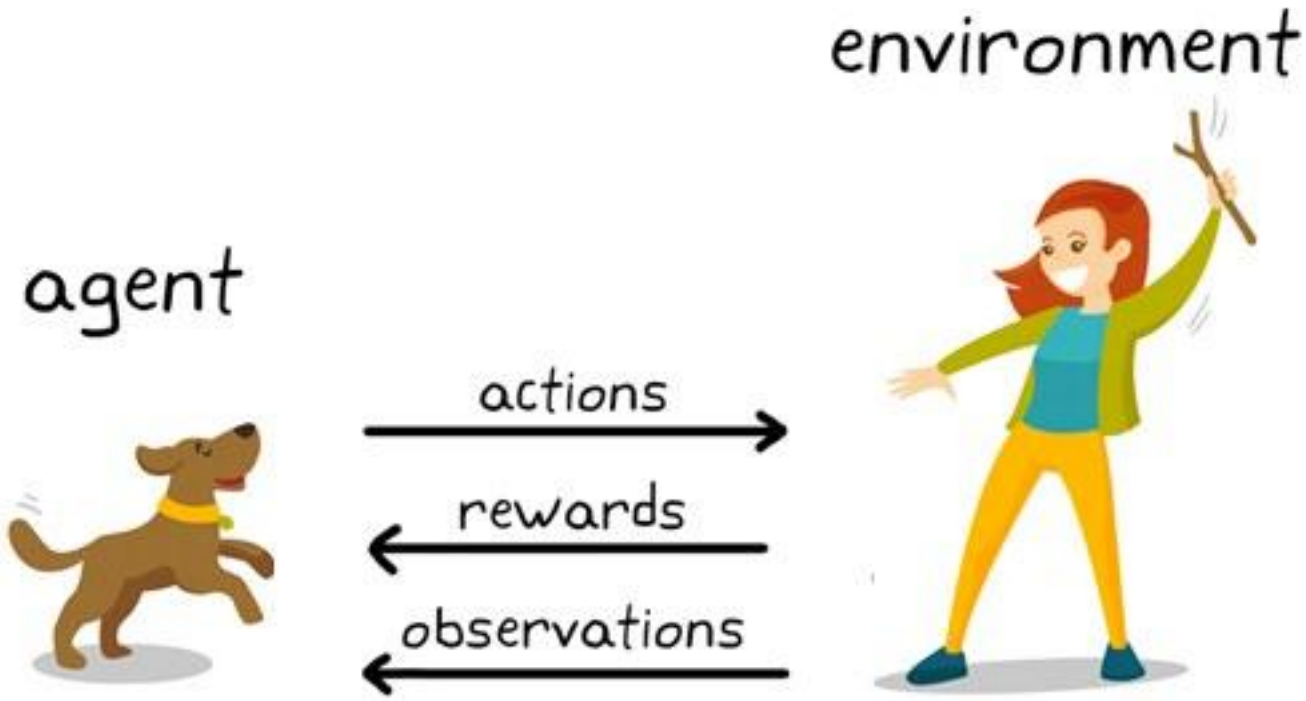
- You have 500 labeled cat vs. dog images.
- And 10,000 unlabeled images.
- **Steps**:
- Train a classifier on 500 labeled images.
- Predict on 10,000 unlabeled images.
- For predictions with high confidence (e.g., 95%), assign pseudo-labels.
- Combine pseudo-labeled + labeled data.
- Retrain the model with this larger dataset.

Introduction to Reinforcement Learning

What is observation from image ?

Introduction to Reinforcement Learning

Each time the dog gets a stick successfully, you offered him a feast (a bone let's say). Eventually, the dog understands the pattern, that whenever the master throws a stick, it should get it as early as it can to gain a reward (a bone) from a master in a lesser time.

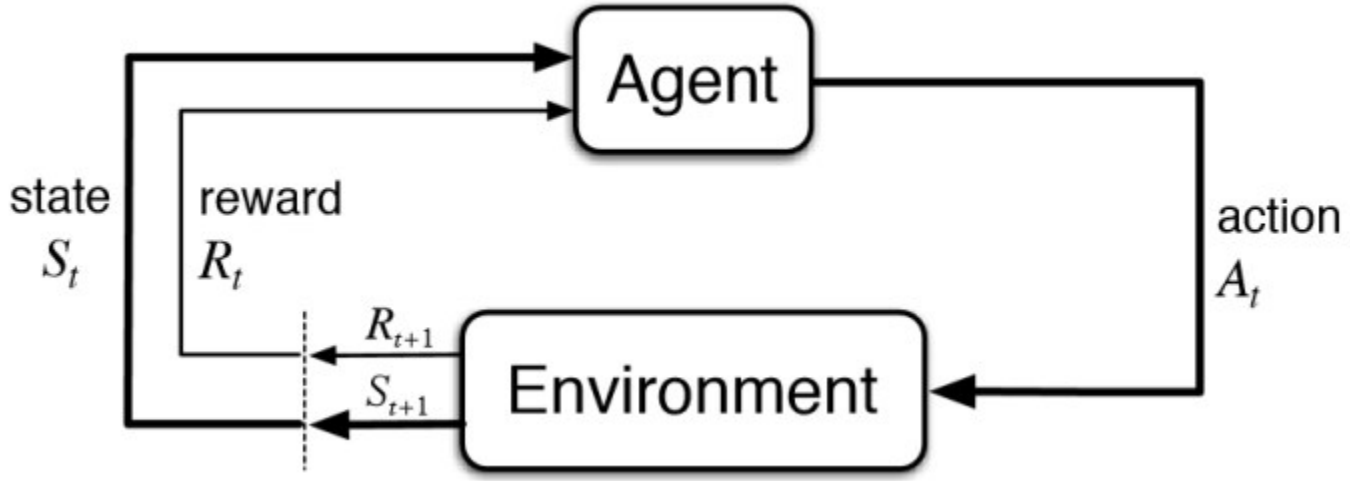


Practical Applications of reinforcement learning

- Robotics for Industrial Automation
- Text summarization engines, dialogue agents (text, speech), gameplays
- Autonomous Self Driving Cars
- Machine Learning and Data Processing
- Training system which would issue custom instructions and materials with respect to the requirements of students
- AI Toolkits, Manufacturing, Automotive, Healthcare, and Bots
- Aircraft Control and Robot Motion Control
- Building artificial intelligence for computer games

Introduction to Reinforcement Learning

Reinforcement learning, a type of machine learning, in which agents take actions in an environment aimed at maximizing their cumulative rewards – NVIDIA

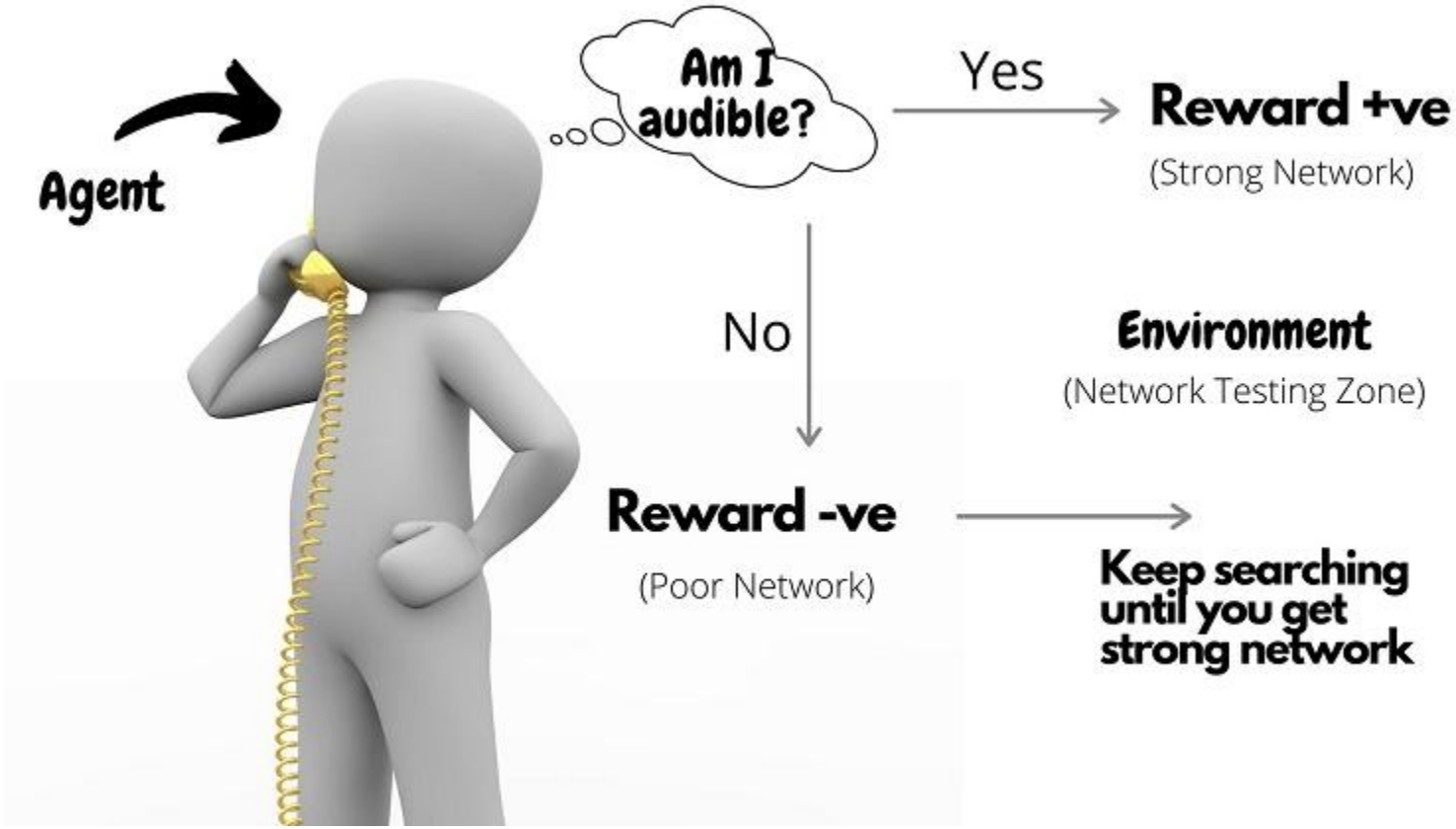


It is a type of machine learning technique where a computer agent learns to perform a task through repeated trial and error interactions with a dynamic environment. Eg. Games This learning approach enables the agent to make a series of decisions that maximize a reward metric for the task without human intervention and without being explicitly programmed to achieve the task

Introduction to Reinforcement Learning

Students in Zoom Meeting ?

Faculty learns to teach without response



Definition

- Reinforcement learning, a type of machine learning, in which agents take actions in an environment aimed at maximizing their cumulative rewards – NVIDIA

- Reinforcement learning (RL) is based on rewarding desired behaviors or punishing undesired ones. Instead of one input producing one output, the algorithm produces a variety of outputs and is trained to select the right one based on certain variables – Gartner

- It is a type of machine learning technique where a computer agent learns to perform a task through repeated trial and error interactions with a dynamic environment. This learning approach enables the agent to make a series of decisions that maximize a reward metric for the task without human intervention and without being explicitly programmed to achieve the task – Mathworks

Terminologies

- **Agent** – is the sole decision-maker and learner
- **Environment** – a physical world where an agent learns and decides the actions to be performed
- **Action** – a list of action which an agent can perform
- **State** – the current situation of the agent in the environment
- **Reward** – For each selected action by agent, the environment gives a reward. It's usually a scalar value and nothing but feedback from the environment
- **Policy** – the agent prepares strategy(decision-making) to map situations to actions.
- **Value Function** – The value of state shows up the reward achieved starting from the state until the policy is executed
- **Model** – Every RL agent doesn't use a model of its environment. The agent's view maps state-action pairs probability distributions over the states

Reinforcement Learning Workflow

1. **Set up the environment and agent**
   → Create the world where the agent will learn, and initialize the agent with no knowledge.
2. **Start an episode**
   → Begin one round of learning (like playing one game).
3. **Observe the current state**
   → The agent looks at where it is or what's happening.
4. **Choose an action**
   → Based on what it knows so far, the agent picks an action (like move left, jump, or pick something).
5. **Perform the action**
   → The action is carried out in the environment.

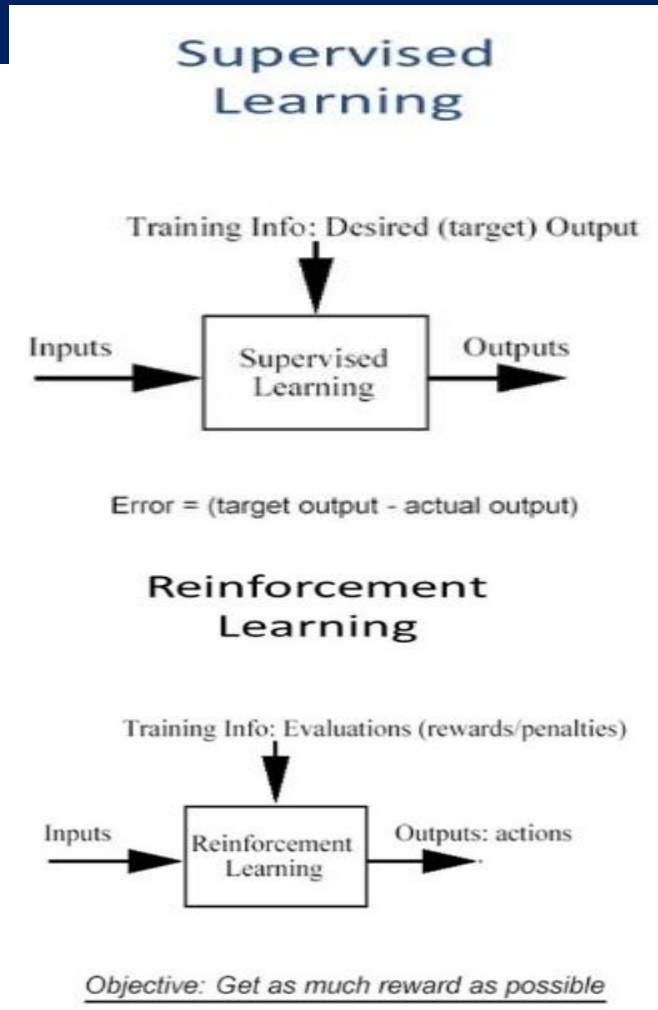
Reinforcement Learning Workflow

6. Receive reward and new state
   → The environment gives the agent feedback:
   ✓ A reward (good or bad)
   ➕ A new state (the result of the action)
7. Learn from the experience
   → The agent updates what it has learned so it can make better choices next time.
8. Repeat steps 3–7
   → Keep doing this until the task or episode is finished.
9. Start next episode
   → Begin a new round and try to do better based on past learning.
10. Keep training until the agent gets good at the task
    → After many episodes, the agent learns the best way to act to get the most reward.

How is reinforcement learning different from supervised learning?

In supervised learning, the model is trained with a training dataset that has a correct answer key. The decision is done on the initial input given as it has all the data that's required to train the machine. The decisions are independent of each other so each decision is represented through a label. Example: Object Recognition

In reinforcement learning, there isn't any answer and the reinforcement agent decides what to be done to perform the required task. the agent had to learn from its experience. It's all about compiling the decisions in a sequential manner.

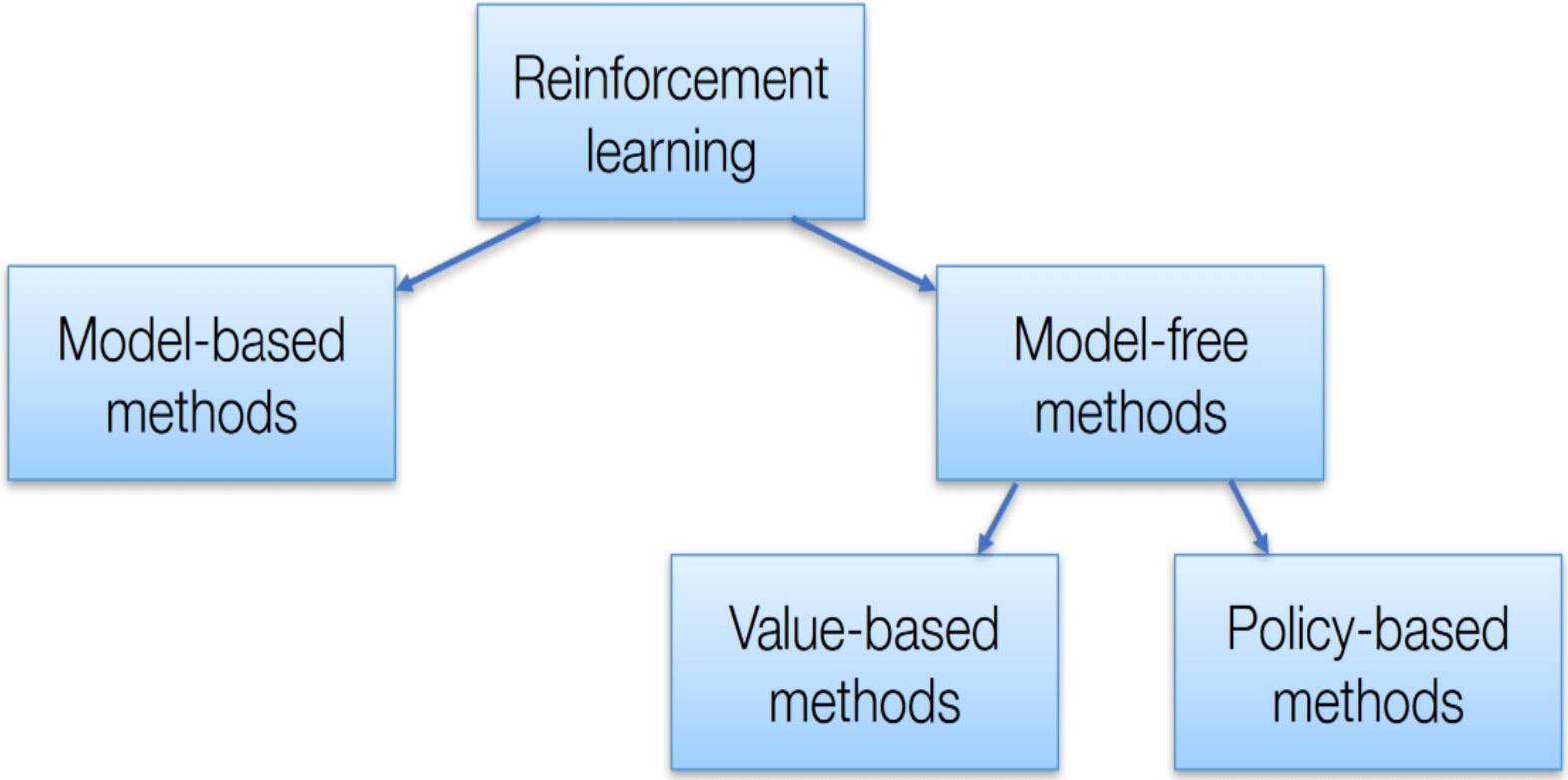


Characteristics of Reinforcement Learning

- No supervision, only a real value or reward signal
- Decision making is sequential
- Time plays a major role in reinforcement problems
- Feedback isn't prompt but delayed

Reinforcement Learning Algorithms

There are 3 approaches to implement reinforcement learning algorithms



Reinforcement Learning Algorithms

- **Value-Based** – The main goal of this method is to maximize a value function. Here, an agent through a policy expects a long-term return of the current states.
- **Policy-Based** – In policy-based, you enable to come up with a strategy that helps to gain maximum rewards in the future through possible actions performed in each state. Two types of policy-based methods are deterministic and stochastic.
- **Model-Based** – In this method, we need to create a virtual model for the agent to help in learning to perform in each specific environment

Types of Reinforcement Learning

- **1. Positive Reinforcement**
- Positive reinforcement is defined as when an event, occurs due to specific behavior, increases the strength and frequency of the behavior. It has a positive impact on behavior.
- Advantages
- – Maximizes the performance of an action
- – Sustain change for a longer period
- Disadvantage
- – Excess reinforcement can lead to an overload of states which would minimize the results.
-

Types of Reinforcement Learning

2. Negative Reinforcement

Negative Reinforcement is represented as the strengthening of a behavior. In other ways, when a negative condition is barred or avoided, it tries to stop this action in the future.

Advantages

– Maximized behavior

– Provide a decent to minimum standard of performance

Disadvantage

– It just limits itself enough to meet up a minimum behavior

Models for reinforcement learning

1. **Markov Decision Process (MDP's)**

MDP are mathematical frameworks for mapping solutions in RL.

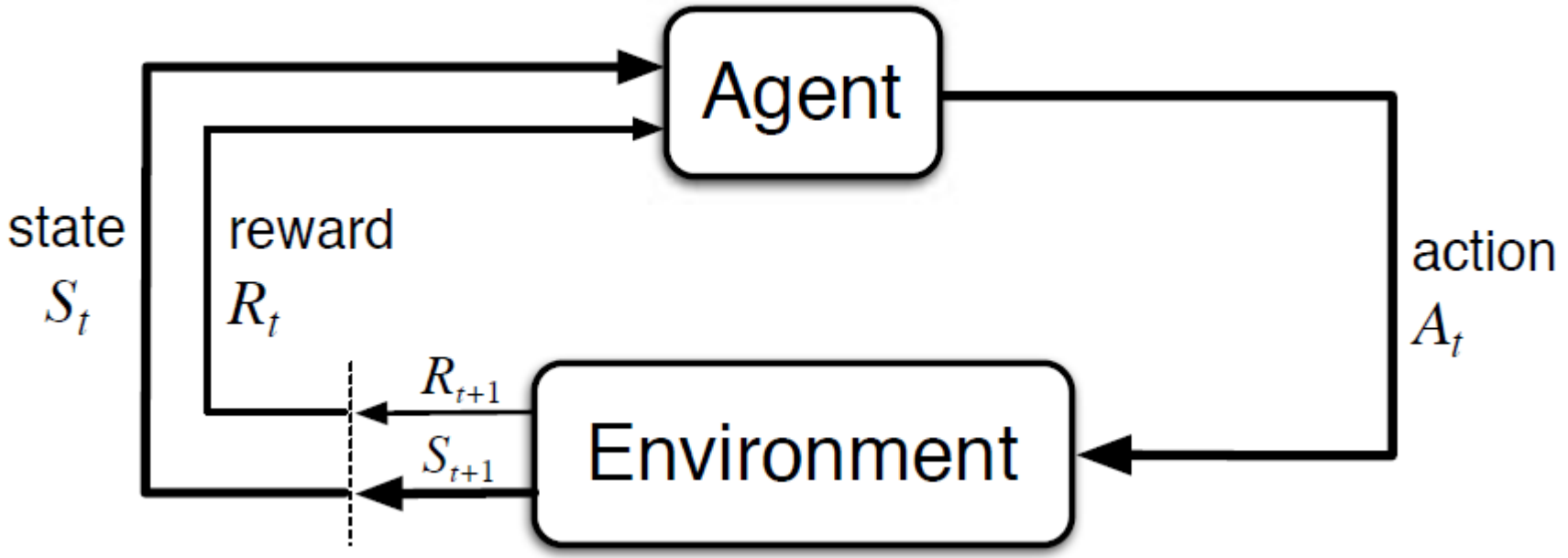
The set of parameters that include

- Set of finite states – S,
- Set of possible Actions in each state – A,
- Reward – R,
- Model – T,
- Policy – π.

The outcome of deploying an action to a state doesn't depend on previous actions or states but on current action and state.

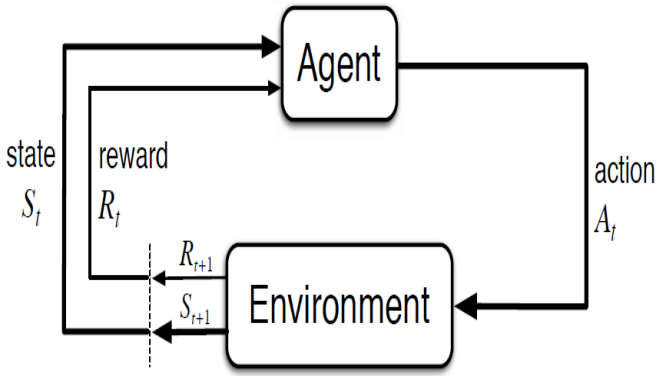
MDP- Markov Decision Process

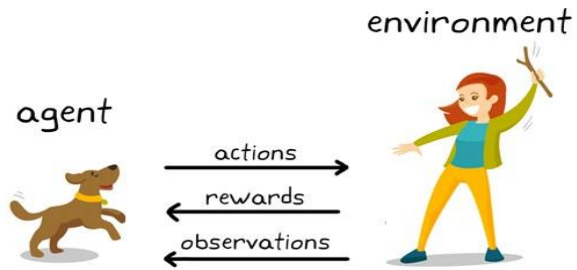
- Markov Decision Process (MDP) is a mathematical framework to describe an environment in reinforcement learning. The following figure shows agent-environment interaction in MDP:



The set of parameters that include
Set of finite states – S_t,
Set of possible Actions in each state – A_t
Reward – R_t,
Model – T,
Policy – π.

MDP- Markov Decision Process





- The agent and the environment interact at each discrete time step, t = 0, 1, 2, 3…At each time step, the agent gets information about the environment state S_t.
- Based on the environment state at instant t, the agent chooses an action A_t.
- the agent also receives a numerical reward signal R_{t+1}.
- This gives rise to a sequence like $S_0, A_0, R_1, S_1, A_1, R_2$…
- The random variables R_t and S_t have well defined discrete probability distributions.

MDP- Markov Decision Process

- The random variables R_t and S_t have well defined discrete probability distributions.
- These probability distributions are dependent only on the preceding state and action by virtue of Markov Property.
- Let S, A, and R be the sets of states, actions, and rewards.
- Then the probability that the values of S_t, R_t and A_t taking values s', r and a with previous state s is given by,

$$p(s', r|s, a) = P\{S_t = s', R_t = r \,|S_{t-1} = s, A_{t-1} = a\}$$

The function p controls the dynamics of the process.

Markov Property

The state that :
"Future is Independent of the past given the present"
Mathematically we can express this statement as :

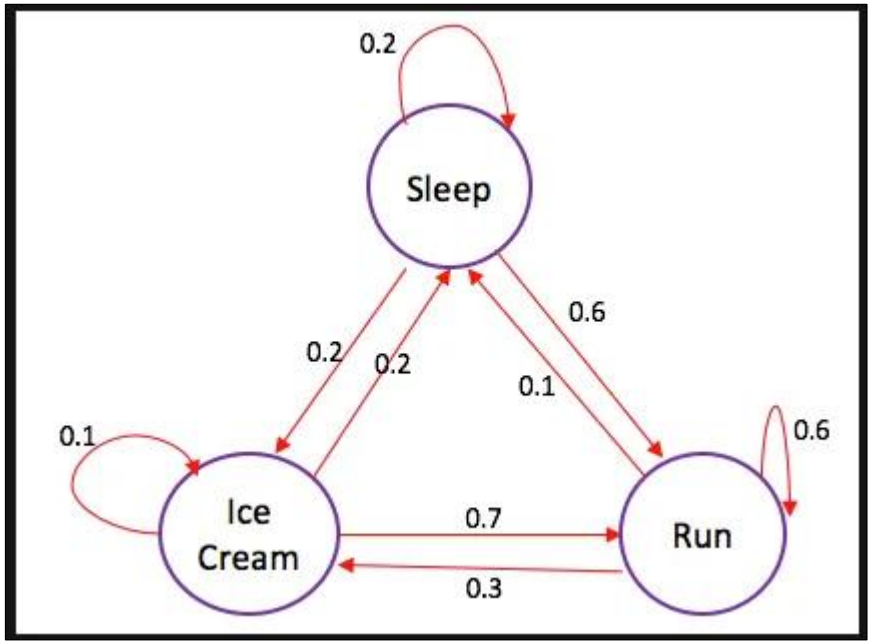
$$P[S_{t+1} \mid S_t] = P[S_{t+1} \mid S_1 , \ldots , S_t]$$

S[t] denotes the current state of the agent and s[t+1] denotes the next state. Intuitively meaning that our current state already captures the information of the past states.

Markov Process or Markov Chains

The edges of the tree denote **transition probability**.

suppose that we were sleeping and then according to the probability distribution there is a **0.6** chance that we will **Run** and **0.2** chance we **sleep more** and again **0.2** that we will eat **ice-cream**.

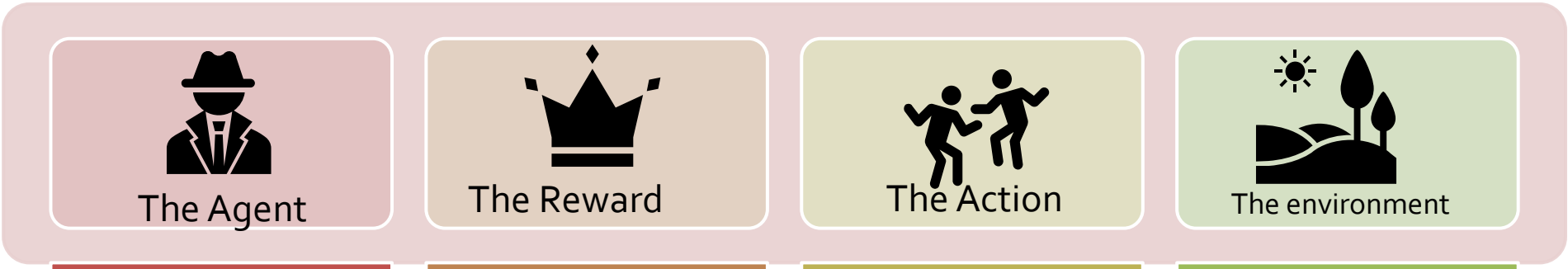


Some samples from the chain :
•Sleep — Run — Ice-cream — Sleep
•Sleep — Ice-cream — Ice-cream — Run

In the above two sequences we get random set of States(S) (i.e. Sleep,Ice-cream,Sleep) every time we run the chain. So Markov process is called random set of sequences.

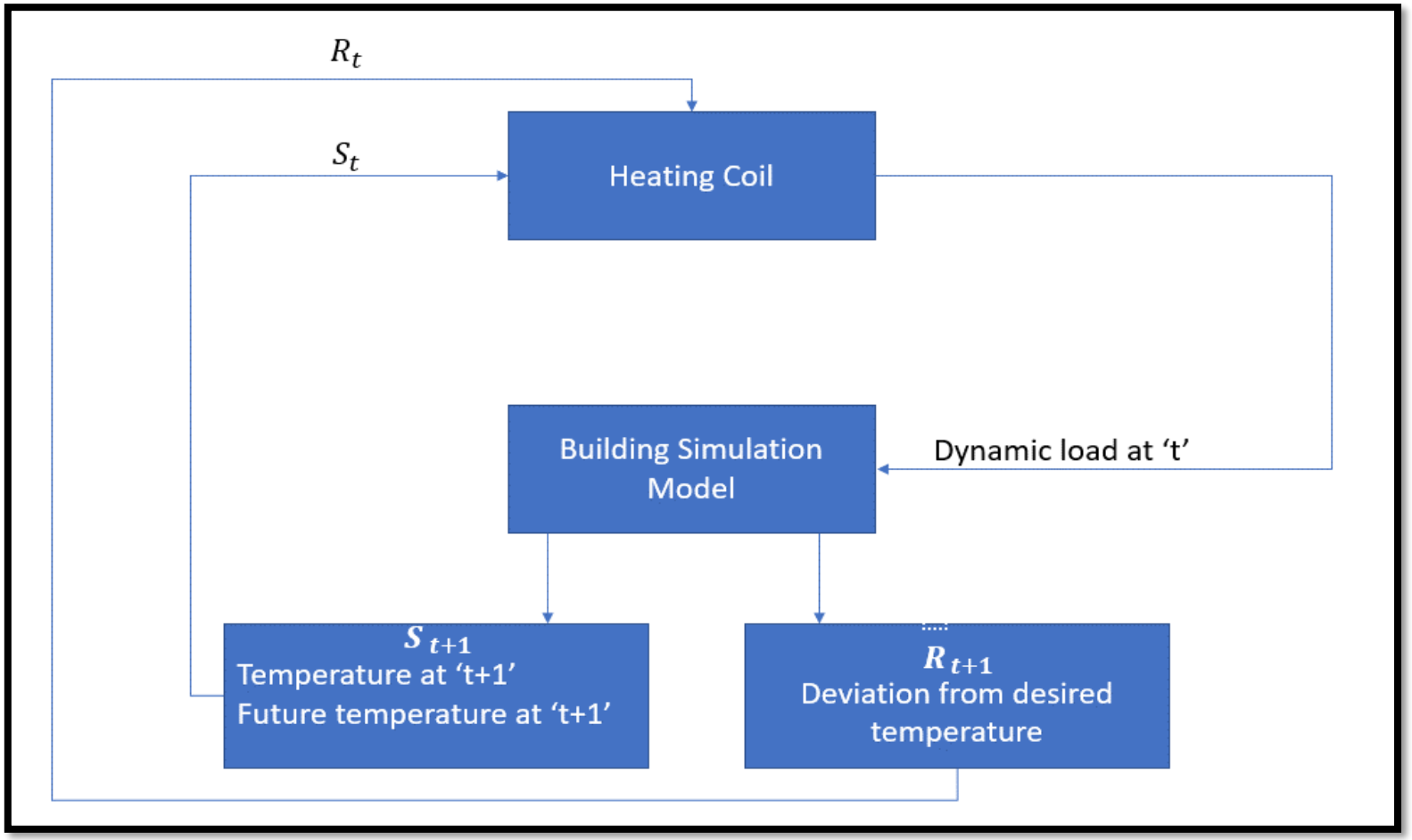
Use Case

- Problem : To control the temperature of a room within the specified temperature limits.



Use Case

Block diagram explains how MDP can be used for controlling the temperature inside a room



Reward and Returns

- **Rewards** are the **numerical values** that the agent receives on performing some action at some **state(s)** in the environment. The numerical value can be positive or negative based on the actions of the agent. The **total sum of reward** the agent receives from the environment is called **returns**.
- We can define Returns as :

$$G_t = r_{t+1} + r_{t+2} + \ldots\ldots + r_T$$

  - $r[t+1]$ is the reward received by the agent at time step t[0] while performing an action(a) to move from one state to another.
  - r[t+2] is the reward received by the agent at time step t[1] by performing an action to move to another state.
  - r[T] is the reward received by the agent by at the final time step by performing an action to move to another state.

Episodic and Continuous Tasks

- ***Episodic Tasks**: These are the tasks that have a **terminal state** (end state)*
- ***Continuous Tasks** : These are the tasks that have no ends i.e. they **don't have any terminal state**. These types of tasks will **never end**.*
- it's easy to calculate the returns from the episodic tasks as they will eventually end but what about continuous tasks, as it will go on and on forever. The returns from sum up to infinity. To solve this difficulty Discount factor is defined

Discount Factor (ɣ):

- ***Discount Factor (ɣ):*** It determines how much **importance** is to be given to the **immediate reward and future rewards**.
- This basically helps us to avoid **infinity** as a reward in continuous tasks.
- It has a value between 0 and 1.
- A value of **0** means that more importance is given to the **immediate reward** and a value of **1** means that more importance is given to **future rewards**. Formula for returns using discount factor

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

Example

- suppose you live at a place where you face water scarcity so if someone comes to you and say that he will give you 100 liters of water for the next 15 hours as a function of some parameter (**ɣ**).
- **Case 1:** discount factor (**ɣ**) **0.8 :**

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

$$G_t = 100 + 80 + 64 \ldots$$

This means that we should wait till 15th hour because the decrease is not very significant. In 15th hour also some significant amount of water we will get . So **ɣ** near to 1 gives future rewards. Another example is Chess where after few moves the King is defeated

Example

- **Case 1:** discount factor (**ɣ**) **0.2 :**

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

$$G_t = 100 + 20 + 4 \ldots$$

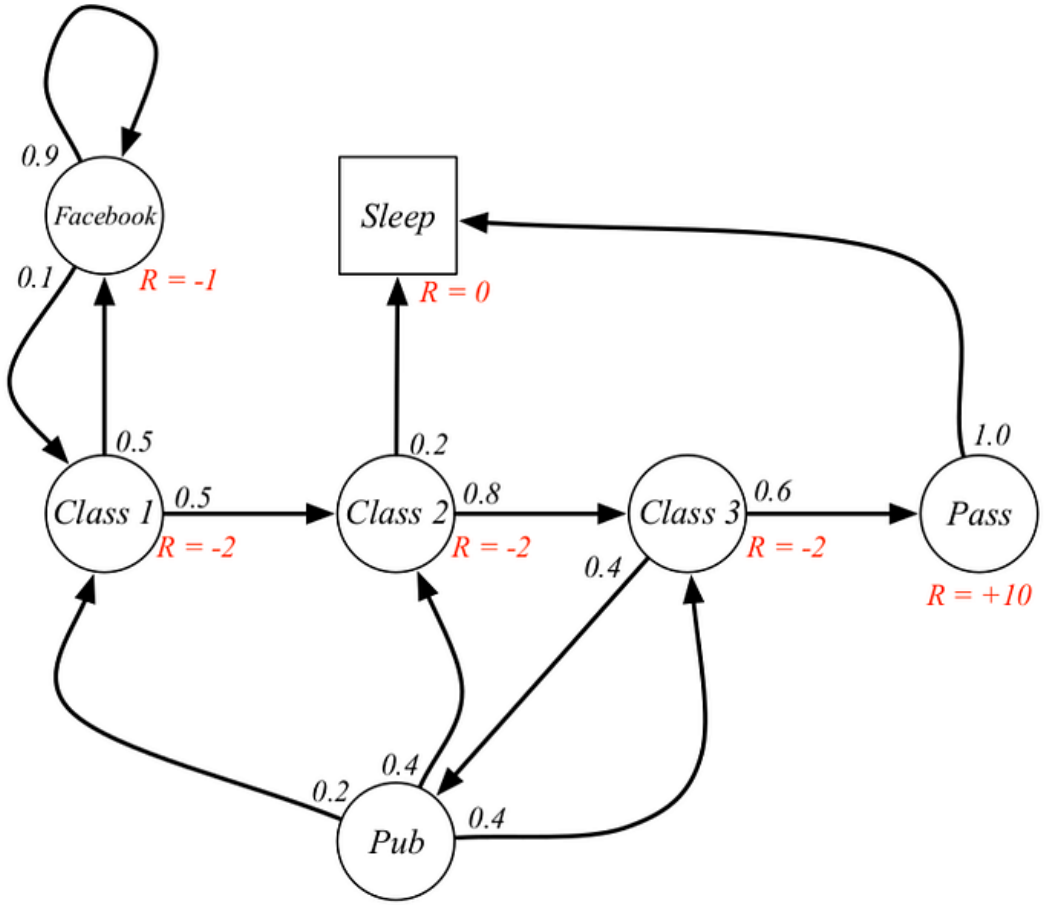
we are more interested in early rewards as the rewards are getting significantly low at hour. we might not want to wait till the end (till 15th hour) as it will be worthless. If the discount factor is close to zero then immediate rewards are more important that the future.

Use case :

Suppose our start state is Class 2, and we move to Class 3 then Pass then Sleep
Class 2 > Class 3 > Pass > Sleep. Discount factor Gamma = 0.5

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

-2 + (-2 * 0.5) + 10 * 0.25 + 0
= -0.5
the value of Class 2 is -0.5 .



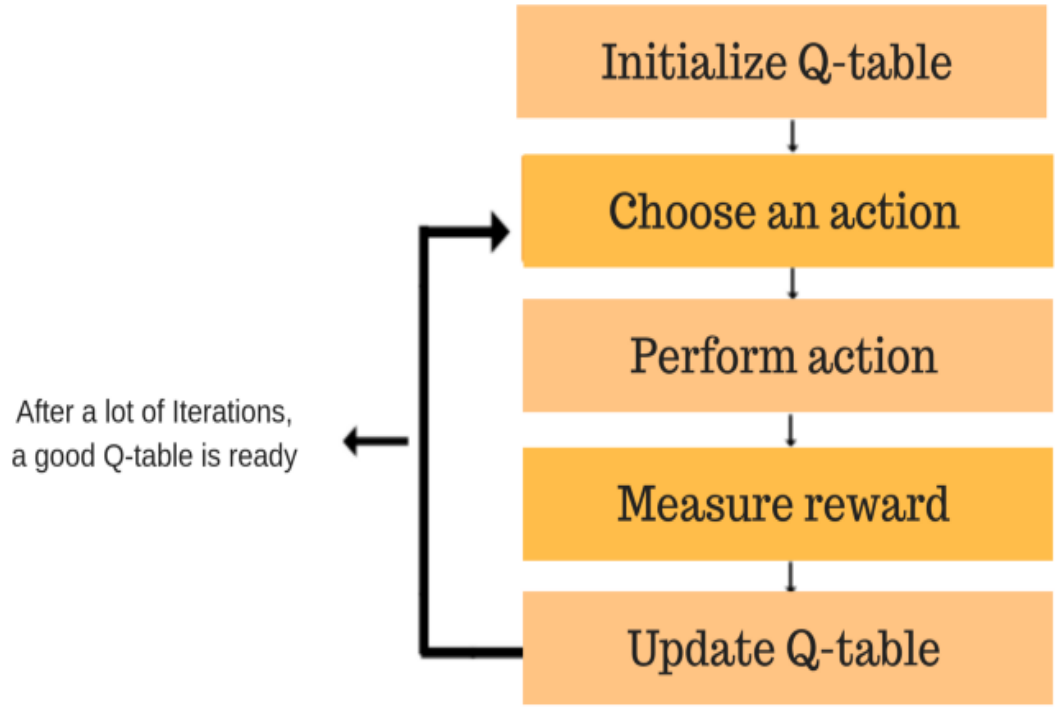
Limitations of this Method

- Reinforcement learning learns from the state. The state is the input for policy making. Hence, the state inputs should be correctly given.
- There are multiple variables and the dimensionality is huge.
- So using it for real physical systems would be difficult!

Q learning

- Q-learning is a model-free, value-based, off-policy learning algorithm.
- **Model-free:** The algorithm that estimates its optimal policy without the need for any transition or reward functions from the environment.
- **Value-based:** Q learning updates its value functions based on equations, (say Bellman equation) rather than estimating the value function with a greedy policy.
- **Off-policy:** The function learns from its own actions and doesn't depend on the current policy.

Q learning


Initialize Q-table
Choose an action
Perform action
Measure reward
Update Q-table
After a lot of Iterations,
a good Q-table is ready

Use Case – Robots in a Warehouse

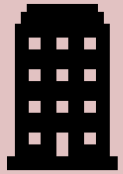
- A growing e-commerce company is building a new warehouse, and the company would like all of the picking operations in the new warehouse to be performed by warehouse robots.
- Requirements are:
  - In the context of e-commerce warehousing, “picking” is the task of gathering individual items from various locations in the warehouse in order to fulfill customer orders.
  - After picking items from the shelves, the robots must bring the items to a specific location within the warehouse where the items can be packaged for shipping.
  - In order to ensure maximum efficiency and productivity, the robots will need to learn the shortest path between the item packaging area and all other locations within the warehouse where the robots are allowed to travel.

USE Case

- https://www.youtube.com/watch?v=LDhJ5I89H_I

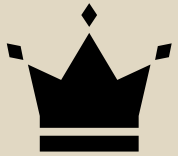
Q learning

- The environment consists of **states**, **actions**, and **rewards**.



The State

The states in the environment are all of the possible locations and The agent's goal is to learn the shortest path



The Reward

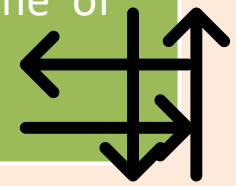
To help the agent learn, each state (location) is assigned a reward

Agent has ***to maximize its total rewards***!

This encourages the agent to identify the shortest path to the goal by minimizing its punishments!

The Action

The actions that are available to the agent are to move the robot in one of four directions:

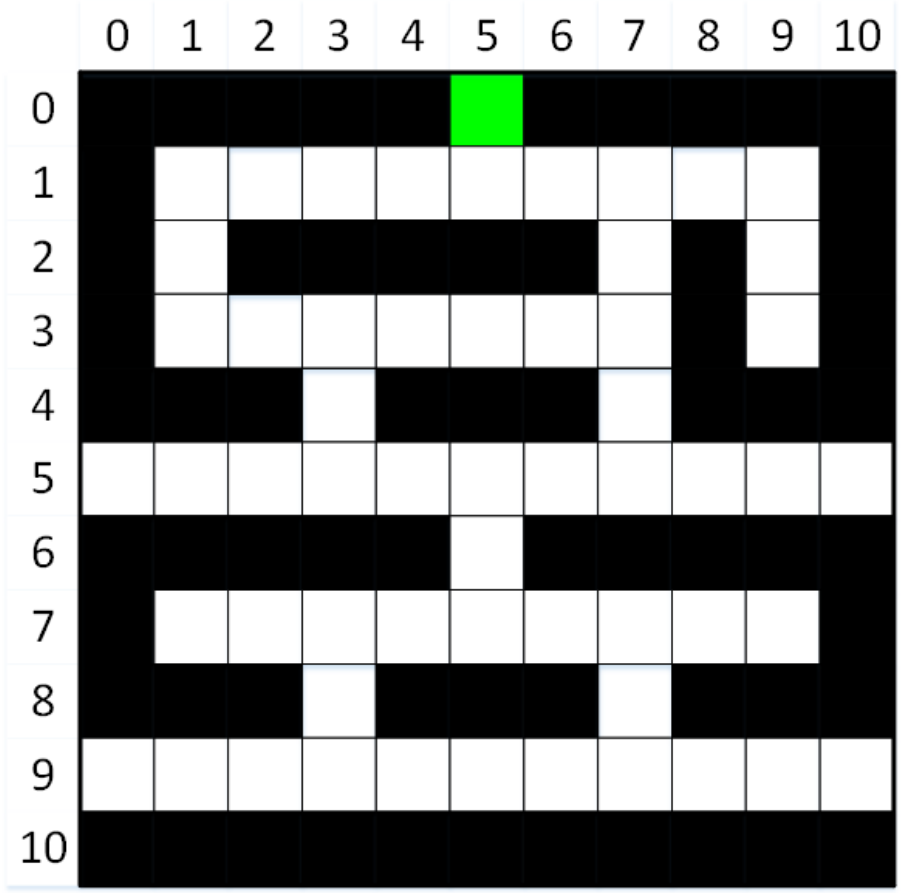


Warehouse is divided into grid /squares

black squares = locations are for storing items

White squares = locations are aisles that the robot can use to travel throughout the warehouse

Green square = The item packaging area



The learning process will follow these steps:

1. Choose a random, non-terminal state (white square) for the agent to begin this new episode.
2. Choose an action (move *up*, *right*, *down*, or *left*) for the current state. Actions will be chosen using an *epsilon greedy algorithm*. This algorithm will usually choose the most promising action for the agent, but it will occasionally choose a less promising option in order to encourage the agent to explore the environment.
3. Perform the chosen action, and transition to the next state (i.e., move to the next location).
4. Receive the reward for moving to the new state, and calculate the temporal difference.
5. Update the Q-value for the previous state and action pair.
6. If the new (current) state is a terminal state,[green square] go to #1. Else, go to #2.

The learning process will follow these steps:

- This entire process will be repeated across 1000 episodes. This will provide the agent sufficient opportunity to learn the shortest paths between the item packaging area and all other locations in the warehouse where the robot is allowed to travel, while simultaneously avoiding crashing into any of the item storage locations

Calculation of shortest Path

Shortest path 1 :
[[3, 9], [2, 9], [1, 9], [1, 8], [1, 7], [1, 6], [1, 5], [0, 5]]

Shortest path 2 :
[[5, 0], [5, 1], [5, 2], [5, 3], [4, 3], [3, 3], [3, 4], [3, 5], [3, 6], [3, 7], [2, 7], [1, 7], [1, 6], [1, 5], [0, 5]]

Shortest path 3 :
[[9, 5], [9, 6], [9, 7], [8, 7], [7, 7], [7, 6], [7, 5], [6, 5], [5, 5], [5, 6], [5, 7], [4, 7], [3, 7], [2, 7], [1, 7], [1, 6], [1, 5], [0, 5]]

robot can currently deliver an item from anywhere in the warehouse ***to*** the packaging area,

	0	1	2	3	4	5	6	7	8	9	10
0	-100	-100	-100	-100	-100	100	-100	-100	-100	-100	-100
1	-100	-1	-1	-1	-1	-1	-1	-1	-1	-1	-100
2	-100	-1	-100	-100	-100	-100	-100	-1	-100	-1	-100
3	-100	-1	-1	-1	-1	-1	-1	-1	-100	-1	-100
4	-100	-100	-100	-1	-100	-100	-100	-1	-100	-100	-100
5	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
6	-100	-100	-100	-100	-100	-1	-100	-100	-100	-100	-100
7	-100	-1	-1	-1	-1	-1	-1	-1	-1	-1	-100
8	-100	-100	-100	-1	-100	-100	-100	-1	-100	-100	-100
9	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
10	-100	-100	-100	-100	-100	-100	-100	-100	-100	-100	-100

Next Item to be Picked Up

- It's great that our robot can automatically take the shortest path from any 'legal' location in the warehouse to the item packaging area. **But what about the opposite scenario?**
- **Packaging Area to other location to pick up next item**
- Solution : **Reversing the order of the shortest path!**

```
#display an example of reversed shortest path:
 path = get_shortest_path(5, 2)
 #go to row 5, column 2 path.reverse()
 print(path)
```

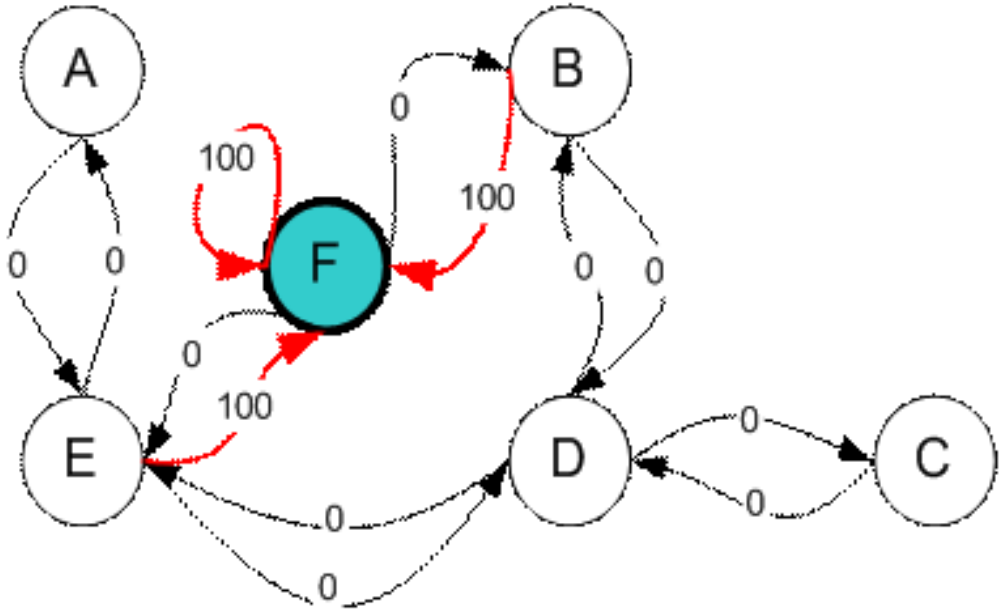
Output:
[[0, 5], [1, 5], [1, 6], [1, 7], [2, 7], [3, 7], [3, 6], [3, 5], [3, 4], [3, 3], [4, 3], [5, 3], [5, 2]]

Use Case 2

- Suppose we have 5 rooms in a building connected by doors as shown in the figure below. We'll number each room from A through E. The outside of the building can be thought of as one big room (F).
- Scenario: put an agent in any room, and from that room, go outside the building (this will be our target room F).
- Take learning factor Y = 0.8
- The Gamma parameter has a range of 0 to 1 (0 <= Gamma > 1). If Gamma is closer to zero, the agent will tend to consider only immediate rewards. If Gamma is closer to one, the agent will consider future rewards with greater weight, willing to delay the reward.

Use Case 2 :

We can represent the rooms on a graph, each room as a node, and each door as a link.
To set this room as a goal, we'll associate a reward value to each door (i.e. link between nodes).
The doors that lead immediately to the goal have an instant reward of 100. Other doors not directly connected to the target room have zero rewards.
doors are two-way (0 leads to 4, and 4 leads back to 0), two arrows are assigned to each room. Each arrow contains an instant reward value, as shown



Q learning

- Now suppose we have an agent in Room B and we want the agent to learn to reach outside the house (F).
- Q-Learning includes the terms “state” and “action”.
- State= each room, including outside,
- Action = The agent’s movement from one room to another will be an “action”.
- In diagram, a “state” is depicted as a node, while “action” is represented by the arrows.

Design of Q matrix

- Q matrix represents the memory of an Agent learned through experience
- The rows of matrix Q represent the current state of the agent, and the columns represent the possible actions leading to the next state (the links between the nodes).
- The agent starts out knowing nothing, the matrix Q is initialized to zero

Formula

- **The transition rule of Q learning is a very simple formula:**
- **Q(state, action) = R(state, action) + Ɣ * Max[Q(next state, all actions)]**
- According to this formula, a value assigned to a specific element of matrix Q is equal to the sum of the corresponding value in matrix R and the learning parameter Gamma, multiplied by the maximum value of Q for all possible actions in the next state.

- virtual agent will learn through experience. The agent will explore from state to state until it reaches the goal. Each exploration is called an episode
- Each episode consists of the agent moving from the initial state to the goal state.

Initial Q matrix – no memory –no experience

	A	B	C	D	E	F
A	o	0	0	0	0	0
B	0	0	0	0	0	0
C	0	0	0	0	0	0
D	0	0	0	0	0	0
E	0	0	0	0	0	0
F	0	0	0	0	0	0

Reward matrix

State/Action	A	B	C	D	E	F
A	--	--	---	--	0	---
B	---	---	---	0	---	100
C	---	---	---	0	---	---
D	---	0	0	---	0	--
E	0	---	---	0	---	100
F	---	0	---	---	0	100

Randomly we start with door B
Look at the second row (state B) of matrix R. There are two possible actions for the current state :
 go to state D,
or go to state F.
By random selection, we select to go to F as our action. It has 3 possible actions: go to states B, E, or F.

- Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]

  Q(B, F) = R(B, F) + 0.8 * Max[Q(F, B), Q(F,E), Q(F, F)] = 100 + 0.8 * 0 = 100
- Initial Q matrix is all zero so
- Q(B, F) = R(B, F) + 0.8 * Max [0]
- = 100+ 0
- = 100
- The next state, F, now becomes the current state.
- Because 5 is the goal state, we've finished one episode.
- Our agent's brain now contains an updated matrix Q as:

Updated Q

	A	B	C	D	E	F
A	o	0	0	0	0	0
B	0	0	0	0	0	100
C	0	0	0	0	0	0
D	0	0	0	0	0	0
E	0	0	0	0	0	0
F	0	0	0	0	0	0

- Now Let us start episode 2 with random start at gate D

Reward matrix

State/Action	A	B	C	D	E	F
A	--	--	---	--	0	---
B	---	---	---	0	---	100
C	---	---	---	0	---	---
D	---	0	0	---	0	--
E	0	---	---	0	---	100
F	---	0	---	---	0	100

- Look at the fourth row of matrix R; it has 3 possible actions: go to states B, C, or E. By random selection, we select to go to state B as our action.
- Now we imagine that we are in state B. Look at the second row of reward matrix R (i.e. state B). It has 2 possible actions: go to state 3 or state 5

Calculate new Q value

- Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]

Q(D, B) = R(D, B) + 0.8 * Max[Q(B, D), Q(B, F)] = 0 + 0.8 * Max(0, 100) = 80

Q(B, F) is calculated 100 in last episode

Q (D,B)= 80 from this episode

Update Q matrix after second episode is

	A	B	C	D	E	F
A	o	0	0	0	0	0
B	0	0	0	0	0	100
C	0	0	0	0	0	0
D	0	80	0	0	0	0
E	0	0	0	0	0	0
F	0	0	0	0	0	0

After 1000 episodes Q might be

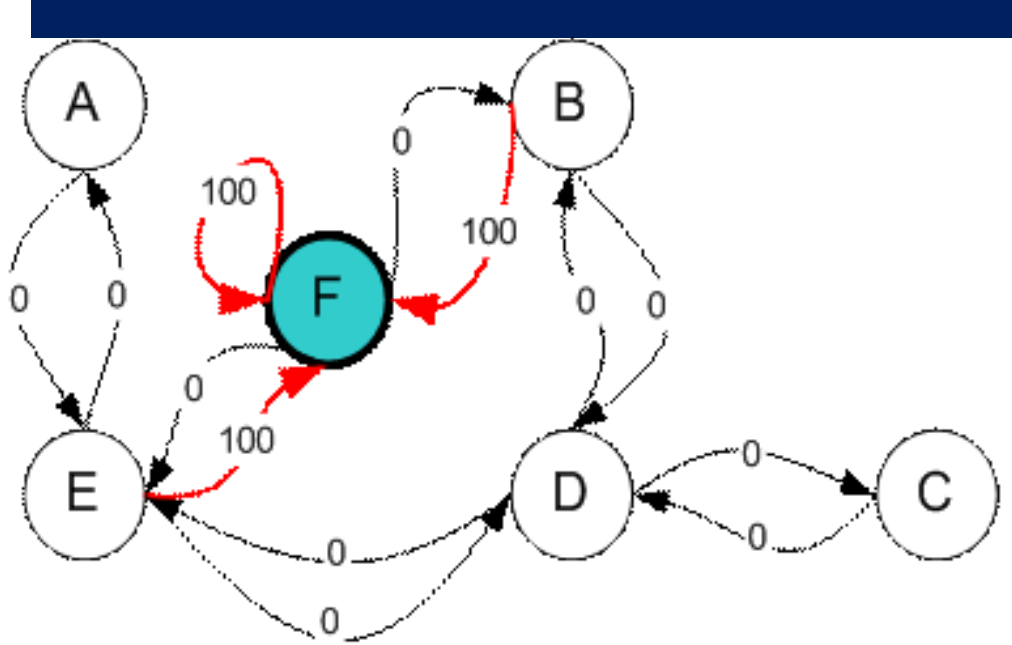
State/Action	A	B	C	D	E	F
A	--	--	---	--	400	---
B	---	---	---	320	---	500
C	---	---	---	320	---	---
D	---	400	256	---	400	--
E	320	---	---	320	---	500
F	---	400	---	---	400	500

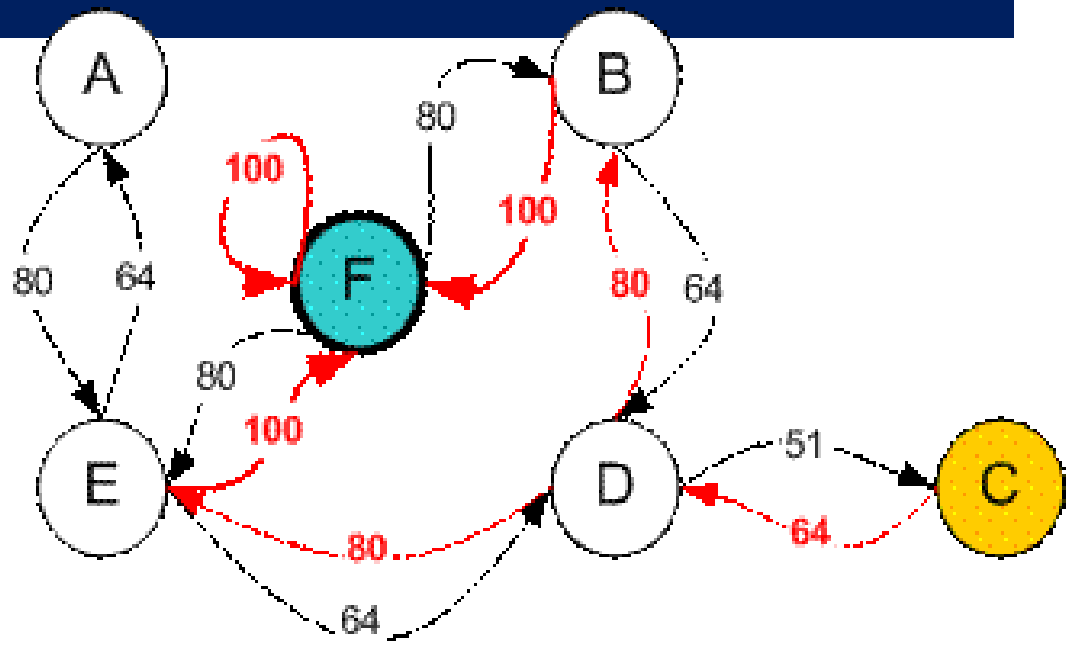
- Convert into percentile / Normalize to get final Q values
- Once the matrix Q gets close enough to a state of convergence, we know our agent has learned the most optimal paths to the goal state.
- Tracing the best sequences of states is as simple as following the links with the highest values at each state.

Convert in percentage / Normalize Q values

State/Action	A	B	C	D	E	F
A	--	--	---	--	80	---
B	---	---	---	64	---	100
C	---	---	---	64	---	---
D	---	80	51	---	80	--
E	64	---	---	64	---	100
F	---	80	---	---	80	100

Agent has learnt shortest paths





- From State C the maximum Q values suggest the action to go to state D.
- From State D the maximum Q values suggest two alternatives: go to state B or E. Suppose we arbitrarily choose to go to B.
- From State B the maximum Q values suggest the action to go to state F.
- Thus the sequence is C – D –B – F .

Thank you